6. 在缓存中一般使用地址的中间位作为组索引，高位作为标签，而不是用高位作为组索引，中间位作为标签。

原因是如果采用地址高位作索引时，有可能将如地址相差一个块大小的两个变量映射到同一组中，若该高速缓存组大小不能容纳多个变量同时存放时，可能会通过多个变量的交替顺序访问来实现相近的结果，发生抖动。

而若采用中位进行索引时，则会将两个变量映射到不同组的高速缓存块中。

可以更好地利用高速缓存的数据存储能力和容量。

7. 地址的组索引与块内偏移的总位数与虚拟内存系统的页偏移位数相同。

好处：缓存系统和虚拟内存系统都是采用了类似的分段分页的思想，缓存系统中的组索引和块内偏移位数用于标识缓存中的数据块，而虚拟内存系统中的页偏移位数用于标识虚拟地址所在页内偏移量。以设计上使缓存系统与虚拟内存系统之间更加协调，提高计算机系统的整体性能。同时这样的方式可以直接将虚拟内存系统的页偏移量对应的存储位置直接转化为对应索引的组中块，以及块内相对偏移位置处。无需多次再次建立一次缓存系统到虚拟内存系统的地址映射算法。

8. 1) L1缓存使用，且程序平均缓存缺失率为3%，则存储系统平均访问延时 $\bar{t}_1 = 0.03\times 110 + 1\times 0.97 = 4.27$ 个周期

2) 可以计算中平均缓存缺失率 $\alpha = \frac{1GB-64KB}{1GB} = 1-\frac{2^6}{2^{20}} = 1-\frac{1}{2^{14}}$

平均访问延时 $\bar{t}_2 = \alpha\times 110 + (1-\alpha)\times 1 = 1+109\alpha = 110-\frac{109}{2^{14}} = 110-0.0067 = 109.9933$ 个周期 近似为110个周期

由于完全随机访问导致缺失率很高。

3) 两者的平均访问延时相差巨大，差别在于(1)问中的局部性原理对于缓存命中的贡献很大，大大降低了平均缓存缺失率。而(2)问中完全随机访问的特点破坏了局部性原理，使得缓存命中率大幅下降，大大延长了平均访问存储系统的时间。

4) 使用L1时获得性能收益，则有 e 表示访问效率

$e = \frac{t_c}{t_a} = \frac{t_c}{h t_c + (1-h) t_m} = \frac{1}{h + (1-h)\frac{t_m}{t_c}}$

t_c：命中时cache访问时间；t_a：cache任存的平均访问时间；t_m：cache缺失时的平均访问时间；h：命中率

有 $t_{平均cache} > t_a$ 即 $105 > h t_c + (1-h) t_m$

$\Rightarrow 105 > h + (1-h)110$

$109h > 5$ $\quad h > \frac{5}{109} \approx 0.0459 = 4.59\%$

即平均缓存命中率高于4.59%即可

9.

编号	地址位数 Bit	缓存大小 KB	块大小 Byte	相联度	组数量	组索引位数 Bit	标签位数 Bit	偏移位数 Bit
1	32	4	64	2	32	5	21	6
2	32	4	64	8	8	3	23	6
3	32	4	64	全相联	1	0	26	6
4	32	16	64	1	256	8	18	6
5	32	16	128	2	64	6	19	7
6	32	64	64	4	256	8	18	6
7	32	64	64	16	64	6	20	6
8	32	64	128	16	32	5	20	7

2KB $2^{11}/2^6$

10.

10. 虽然较大的缓存通常有较低的缺失率，但更大的缓存一般也带来更高的命中延时，最终的性能变化由两者综合决定。假设两个具有L1缓存的存储系统：系统A使用8KB直接映射缓存，命中延时为0.22ns，缓存缺失率为p1；系统B使用64KB四路组相联缓存，命中延时为0.52ns，缓存缺失率为p2。

1) 如果缓存的缺失代价均为额外100ns，则p1和p2满足什么条件时，系统A的平均内存访问时间优于系统B？

2) 如果缓存的缺失代价分别是各系统命中延时的k倍，则p1和p2满足什么条件时，系统A的平均内存访问时间优于系统B？

不妨明确缺失代价为100ns+0.22ns

1) $\bar{t}_A = (1-p_1)\times 0.22 + p_1\times(100+0.22)$ $\quad \bar{t}_A < \bar{t}_B$ 即

$\bar{t}_B = (1-p_2)\times 0.52 + p_2\times(100+0.52)$ $\quad 0.22 + p_1 100 < 0.52 + p_2\times 100$

$p_1 < p_2 + 0.003$

2) $\bar{t}_A = (1-p_1)\times 0.22 + k\,0.22\,p_1$

$\bar{t}_B = (1-p_2)\times 0.52 + p_2\times k\times 0.52$ $\quad \bar{t}_A < \bar{t}_B$

$0.22 + 0.22 p_1 (k-1) < 0.52 + 0.52 p_2 (k-1)$

$p_1 < \frac{15}{11(k-1)} + \frac{26}{11} p_2$

11. 假定数据缓存共有16个块，块大小为64字节。若有一系列发往数据缓存的请求，它们的块地址（块地址是已经去除块内偏移后的内存地址）为：0x1001、0x1005、0x1021、0x1045、0x1305、0x2ee5、0xff05。假设数据缓存初始为空，讨论当此数据缓存为直接映射、2路组相联、4路组相联和8路组相联的情况下，缓存发生块替换的次数。

0101

直接映射时，共有16个块，观察块地址的后4个bit位，相同的有1组，5有4组。

总共发生块替换次数为5次

2路组相联：由块地址后3位bit索引，索引1对应两路填满，索引5对应两路发生3次替换

共有3次替换

4路组相联：由块地址后2位bit索引，索引1对应替换3次 共替换3次

8路组相联 由块地址后1位bit索引 索引1尚未装满，未发生替换。

12. 增加缓存的相联度通常可以降低缺失率，从而提升性能。但对于特殊的程序，则恰恰相反。假设两个块大小为64字节、总容量为256字节的缓存：缓存A为2路组相联，缓存B为直接映射。两个缓存均使用LRU替换策略。分别计算运行下述程序时，这两种缓存的缺失率。(初始时缓存为空，i和j均储存在寄存器中，数组array[]在内存中的起始地址为0)。

```
#define N 100
int32_t array[96];
for(int32_t i = 0; i < N; i++){
    for(int32_t j = 0; j < 96; j++){
        array[j] = i*j;
    }
}
```

不能存16个块 用tag索引

4个块 相联 块内偏移

flag 块地址

$\frac{16+8+16\times 99}{9600} = 16.75\%$

直接映射缓存缺失率

2路组相联：$\frac{16+8+24\times 99}{9600} = 25\%$

根据tag判断是否有符合的块

3位组索引 高位为tag 其中0~7 tag=00

8-15 tag=01

16-23 tag=10

一次循环共要进行24个块大小的写访问

除了开始的块填充外，实现16+8=24次miss

之后的各次访问由于采用了LRU算法均发生了本部分命中

13. 考虑如下所示的C语言代码片段：

```
for(int i = 0; i < 64; ++i){
    for(int j = 0; j < 128; ++j){
        A[j][i] = A[j][i] + 1;
    }
}
```

在内存中数据的组织方式为A[p][q]与A[p][q+1]相邻。请从利用缓存空间局部性的角度，优化以上代码，不可更改代码实现的功能。

可以进行两个for循环的相互调换，变为：

```
for(int j=0; j<128; ++j){
    for(int i=0; i<64; ++i){
        A[j][i]=A[j][i]+1;
    }
}
```

这样以上代码对内存进行访问时，依次按照A[p][q]，A[p][q+1]的方式进行访问。

又因为A[p][q]与A[p][q+1]相邻，对应缓存中存取数据也具有空间上的局部性，可以更好地提高缓存的命中率进而提升性能。

14. 仍然考虑题13中的代码，假设系统仅在对数组A发生读写时访问缓存，其他变量全部位于寄存器中。缓存初始为空。回答以下问题：

1) 假设存在一个4KB大小的直接映射缓存，块大小32字节。则优化前和优化后代码的缓存缺失次数分别为多少？

2) 假设存在一个4KB大小的全相联缓存，块大小32字节，使用FIFO替换策略。则优化前和优化后代码的缓存缺失次数分别为多少？

3) 对于块大小32字节的直接映射缓存，如果希望系统除了必要的强制缺失外不出现任何其他缺失，则优化前和优化后代码分别至少需要多大的缓存容量？

128个块

对于优化前代码，横向遍历有8个块，纵向j遍历列有128个，共计有128×8=1024个块

对于直接映射缓存，

128个 每次进行tag位索引时均发生缺失，缺失次数 2^{13}次 =8192

访问地址 块地址不连续

1) 优化后直接映射缓存：内存中块连续读取，每个块内有8个数据，横向有8个块，有128×8=1024次缺失。

2) 对于FIFO替换全相联，优化前的过程为第一个循环，将128个对应块读入，之后7个连续数据循环，横向读入，故对应共miss 128×8=1024次。

优化后与优化前过程类似，只不过访问相对顺序不同，对结果无影响，也为128×8=1024次缺失

3) 优化前的直接映射，若想把可能的缺失次数降到最小，只需将缓存大小增至可装入所有块数据。

1024个块大小 =1024×32 Byte = 2^{15} Byte

优化后：只需最小一个块就行，由于顺序访问使得可能的miss数均为1024个。

32 Byte

15. 考虑如下所示的实现矩阵转置功能的C语言代码片段：

```
int    input[4][4],output[4][4];
for(int i = 0; i < 4; ++i){
    for(int j = 0; j < 4; ++j){
        output[j][i] = input[i][j];
    }
}
```

假设系统中int变量为4字节，input数组的起始地址为0x0，output数组的起始地址为0x40.假设上述程序运行在一个存在L1缓存的系统中，该缓存总大小32字节，块大小16字节，直接映射，缓存策略为直写、写分配。若初始缓存为空，系统仅在对input和output的读写时会访问缓存。请在下表中填写执行上述程序时数组中每个元素的缓存命中情况。

		input 数组				output 数组			
		列 0	列 1	列 2	列 3	列 0	列 1	列 2	列 3
0	行 0	miss	miss	hit	miss	miss	miss	miss	miss
1	行 1	miss	hit	miss	hit	miss	miss	miss	miss
0	行 2	miss	miss	hit	miss	miss	miss	miss	miss
1	行 3	miss	hit	miss	hit	miss	miss	miss	miss

2块

16. 考虑如下所示的C语言代码片段：

```
int input[2][128];
int sum = 0;
for(int i=0; i<128; ++i){
    sum += input[0][i] * input[1][i];
}
```

}

假设系统中int变量为4字节，input数组的起始地址为0x0。系统仅在对input的读写时会访问缓存，其他变量全部位于寄存器中。则：

1) 如果存在一个512字节大小的缓存，该缓存两路组相联，块大小为16字节，使用LRU替换策略，缓存初始为空。则执行上述程序时缓存的命中率为多少？

2) 在其他条件不变的情况下，增加缓存的总大小可以改善该程序的命中率吗？请说明理由。

3) 在其他条件不变的情况下，增加缓存的块大小可以改善该程序的命中率吗？请说明理由。

64个块 512/16=32

32个块 每路16个块 4个数

相当于两路交叉向量共写，都满足先写1个数据miss，后3个数据hit

命中率 = 3/4 = 75%

不可以 缓存总大小不影响

可以

会增加N的值从而影响命中率 $\frac{N-1}{N}$ 会增大。

1个块中读的数的个数从而可实现提高命中率，命中率 $= \frac{N-1}{N}$

N为一个块中容纳的数据数量。